

Web Development Foundations

Sample / Practice Question Paper – 3

Complete Question Paper + Model Solutions + Exam Answer Explanation Odd Semester 2026–27

Program (Branch), Year, Semester:

Subject Code & Subject Name:

Time: 2 Hours

Maximum Marks: 30

Instructions for Students

1. Attempt all questions.
2. Write syntactically correct HTML/CSS wherever required.
3. Use meaningful element, class and id names.
4. Assume reasonable values wherever necessary.
5. For code-based questions, write the essential code clearly.
6. If the question asks for an explanation, write a short explanation after the code.
7. Do not write unnecessary theory; explain the properties that are relevant to the question.

SECTION – A

Attempt All Questions 5 × 3 = 15 Marks

Q1. HTML Lists and Table – 3 Marks

A university department wants to show its **weekly lab schedule**. Create HTML that contains: An ordered list of three laboratory rules. A table with columns: **Day, Lab, Time and Instructor**. At least three rows of schedule data. Use suitable table elements such as **table, tr, th and td**.

Q2. HTML Multimedia – 3 Marks

Create the essential HTML for a **University Introduction** section containing: A heading and short paragraph. An image with meaningful **alt** text. An audio player with controls. A video player with controls and a suitable width. Also add a link that opens the university website in a new browser tab.

Q3. CSS Display Property – 3 Marks

Three navigation items are written as separate `<a>` elements. Write CSS to:

- (a) Make the links appear as separate rectangular menu items with padding and width.
- (b) Make the three items appear in a vertical menu.
- (c) Name the CSS property/value responsible for this behavior.

Q4. Flexbox Alignment – 3 Marks

A login panel contains a logo, a heading and a button. Write CSS using **Flexbox** so that: The items are arranged vertically. The items are centered horizontally. There is a 15px gap between the items. Mention the main Flexbox properties used.

Q5. CSS Pseudo-classes – 3 Marks

A webpage contains navigation links and a submit button. Write CSS to: Change the link appearance when the mouse is placed over it. Change the button appearance when the mouse is placed over it. Give the first item in a list a different text style. Use appropriate CSS pseudo-classes and briefly state what each pseudo-class does.

SECTION – B

Attempt All Questions 3 × 5 = 15 Marks

Q6. Complete HTML Page – 5 Marks

Create the essential HTML structure for a **Student Achievement webpage**. It must contain: A semantic header containing the page title. A navigation area with four links. A main section containing an achievement introduction. An unordered list of student achievements. An image with meaningful alternative text. An embedded video describing the achievement. A table containing at least three achievements with student name, achievement and year. A footer. Use semantic HTML elements and meaningful attributes.

Q7. Flexbox-Based Page Layout – 5 Marks

Consider this HTML:

```
<div class="page">
  <header>Campus Activities</header>
  <nav>
    <a href="#">Home</a>
    <a href="#">Clubs</a>
    <a href="#">Events</a>
  </nav>

  <main>
    <div class="activity">Coding</div>
    <div class="activity">Music</div>
    <div class="activity">Sports</div>
  </main>

  <footer>University Campus</footer>
</div>
```

Write CSS to:

- (a) Arrange the navigation links horizontally using Flexbox.
- (b) Center the navigation links and provide a 20px gap.
- (c) Arrange the three activity boxes in one horizontal row using Flexbox.
- (d) Give each activity box equal available space, padding, border and margin.
- (e) Add a simple hover effect to the activity boxes using a transition.

Explain the purpose of the major properties used.

Q8. Integrated HTML + CSS Task – 5 Marks

A **personal student portfolio** contains a profile heading, navigation links, an about section, skills list, project information and a contact button. Write essential HTML and CSS to: Create a semantic page structure. Style the navigation links as a horizontal menu using Flexbox. Display the skills as a clean list with suitable spacing. Style the contact button and add a hover state. Use a media query so that the navigation becomes vertical below **600px**. State the purpose of the main CSS properties used.

MODEL SOLUTIONS – HOW TO WRITE IN THE EXAM

Important: Students should first write the required code. When the question asks for an explanation, write the short explanation shown after the code. The explanation should directly relate to the properties used.

Q1 Solution – HTML Lists and Table

```
<h2>Weekly Lab Schedule</h2>

<ol>
  <li>Wear your ID card.</li>
  <li>Follow lab instructions.</li>
  <li>Save your work before leaving.</li>
</ol>

<table border="1">
  <tr>
    <th>Day</th>
    <th>Lab</th>
    <th>Time</th>
    <th>Instructor</th>
  </tr>
  <tr>
    <td>Monday</td>
    <td>Web Lab</td>
    <td>10 AM</td>
    <td>Mr. Sharma</td>
  </tr>
  <tr>
    <td>Wednesday</td>
    <td>Java Lab</td>
    <td>12 PM</td>
    <td>Ms. Singh</td>
  </tr>
  <tr>
    <td>Friday</td>
    <td>Database Lab</td>
    <td>2 PM</td>
    <td>Mr. Verma</td>
  </tr>
</table>
```

What to write after the code: The **ol** element creates a numbered list and **li** defines each list item. The **table** element creates the table, **tr** creates a row, **th** creates heading cells and **td** creates normal data cells.

Answer-writing tip: For a 3-mark HTML question, students do not need a long explanation unless the question asks for one. A short explanation of the important elements is sufficient.

Q2 Solution – HTML Multimedia

```
<h2>About Our University</h2>
<p>Our university provides modern education and facilities.</p>



<audio controls>
  <source src="university-intro.mp3" type="audio/mpeg">
</audio>

<video controls width="500">
  <source src="university-tour.mp4" type="video/mp4">
</video>

<a href="https://example.com" target="_blank">
  Visit University Website
</a>
```

What to write after the code: The **img** element displays the image and the **alt** attribute provides alternative text. The **audio** and **video** elements embed multimedia, while **controls** displays playback controls. The **target="_blank"** attribute opens the link in a new tab.

Q3 Solution – CSS Display Property

```
nav a {
  display: block;
  width: 150px;
  padding: 10px;
  margin: 5px;
  border: 1px solid black;
```

```
text-decoration: none;
}
```

What to write after the code: **display: block** makes each link behave as a block-level element, so each link starts on a new line. The **width** and **padding** make the links look like rectangular menu items. **margin** provides space between the items.

Q4 Solution – Flexbox Alignment

```
.login-panel {
  display: flex;
  flex-direction: column;
  align-items: center;
  gap: 15px;
}
```

What to write after the code: **display: flex** activates Flexbox. **flex-direction: column** arranges the logo, heading and button vertically. **align-items: center** centers the items horizontally. **gap: 15px** creates 15px spacing between the items.

Q5 Solution – CSS Pseudo-classes

```
a:hover {
  color: red;
}

button:hover {
  background-color: black;
  color: white;
}

li:first-child {
  font-weight: bold;
}
```

What to write after the code: The **:hover** pseudo-class applies styles when the mouse pointer is placed over an element. **:first-child** selects an element that is the first child of its parent. Here it is used to style the first list item.

Q6 Solution – Complete Student Achievement Page

```
<!DOCTYPE html>
<html>
<head>
  <title>Student Achievements</title>
</head>
<body>

<header>
  <h1>Student Achievement Portal</h1>
</header>

<nav>
  <a href="#home">Home</a>
  <a href="#achievements">Achievements</a>
  <a href="#projects">Projects</a>
  <a href="#contact">Contact</a>
</nav>

<main>
  <section id="achievements">
    <h2>Our Achievers</h2>
    <p>Our students participate in academic and technical events.</p>

    <ul>
      <li>Won inter-college coding competition</li>
      <li>Secured first position in debate</li>
      <li>Won university sports championship</li>
    </ul>

    

    <video controls width="500">
      <source src="achievement.mp4" type="video/mp4">
    </video>

    <table border="1">
      <tr>
        <th>Student</th>
        <th>Achievement</th>
        <th>Year</th>
      </tr>
      <tr>
        <td>Rahul</td>
        <td>Coding Competition</td>
        <td>2026</td>
      </tr>
      <tr>
        <td>Priya</td>
        <td>Debate Competition</td>
        <td>2026</td>
      </tr>
      <tr>
        <td>Aman</td>
        <td>Sports Championship</td>
        <td>2026</td>
      </tr>
    </table>
  </section>
</main>

<footer>
  <p>© University</p>
</footer>

</body>
</html>
```

What to write after the code: The page uses semantic elements such as **header**, **nav**, **main**, **section** and **footer** to provide a meaningful structure. The **ul/li** elements display achievements as a list. The **img** element displays an image with meaningful **alt** text. The **video** element embeds the achievement video. The **table** displays achievement information in rows and columns.

How to approach a 5-mark scenario question: First make a checklist of every item mentioned in the question. Then build the HTML from the outer structure to the inner content. Before finishing, check that every requested element appears in your code.

Q7 Solution – Flexbox-Based Page Layout

```
.page {
  border: 1px solid black;
```

```

}

nav {
  display: flex;
  justify-content: center;
  gap: 20px;
  padding: 10px;
}

main {
  display: flex;
  gap: 15px;
  padding: 20px;
}

.activity {
  flex: 1;
  padding: 20px;
  margin: 5px;
  border: 1px solid black;
  transition: transform 0.3s;
}

.activity:hover {
  transform: translateY(-5px);
}

```

What to write after the code: **display: flex** makes the navigation and main containers Flexbox containers. **justify-content: center** centers the navigation links horizontally. **gap** provides consistent spacing. **flex: 1** allows each activity box to share the available width equally. **padding** creates inner spacing, **margin** creates outer spacing and **border** creates a visible boundary. **transition** makes the hover effect smooth, while **transform: translateY()** moves the card upward.

Marks-oriented answer: Make sure your answer contains all five requested parts. Missing the explanation can lose marks when the question explicitly asks for it.

Q8 Solution – Integrated Student Portfolio

```

<!DOCTYPE html>
<html>
<head>
<style>
body {
  font-family: Arial, sans-serif;
}

nav {
  display: flex;
  justify-content: center;
  gap: 25px;
  padding: 15px;
}

.skills li {
  margin-bottom: 8px;
}

.contact-btn {
  padding: 10px 18px;
  border: none;
  cursor: pointer;
  transition: 0.3s;
}

.contact-btn:hover {
  transform: scale(1.05);
}

@media (max-width: 600px) {
  nav {
    flex-direction: column;
    align-items: center;
  }
}
</style>
</head>

<body>

<header>
  <h1>My Student Portfolio</h1>
</header>

```

```

<nav>
  <a href="#about">About</a>
  <a href="#skills">Skills</a>
  <a href="#projects">Projects</a>
  <a href="#contact">Contact</a>
</nav>

<main>
  <section id="about">
    <h2>About Me</h2>
    <p>I am a first-year student interested in web development.</p>
  </section>

  <section id="skills">
    <h2>Skills</h2>
    <ul class="skills">
      <li>HTML</li>
      <li>CSS</li>
      <li>JavaScript</li>
    </ul>
  </section>

  <section id="projects">
    <h2>Projects</h2>
    <p>My academic web projects.</p>
  </section>

  <section id="contact">
    <button class="contact-btn">Contact Me</button>
  </section>
</main>

</body>
</html>

```

What to write after the code: **display: flex** makes the navigation a Flexbox container. **justify-content: center** centers the links and **gap** adds space between them. The **transition** property makes the button hover effect smooth and **transform: scale()** slightly enlarges it. The **@media** rule applies when the viewport width is 600px or less, and **flex-direction: column** changes the navigation into a vertical layout. **align-items: center** centers the vertical links.

HOW STUDENTS SHOULD WRITE ANSWERS IN THE ACTUAL PAPER

- **For HTML-only questions:** Write the complete required HTML. Add a short explanation only when the question asks for it.
- **For CSS questions:** Write the CSS first, then explain the important properties in 3–6 short points if explanation is requested.
- **For scenario-based 5-mark questions:** Make sure every requirement in the question is visible in your code. Do not skip small requirements such as a footer, alt text, gap, media query or hover effect.
- **For 'State the purpose' questions:** Explain what each major property does, not the entire CSS language.
- **For 'Explain the major layout decisions' questions:** Explain why Grid/Flexbox/positioning/media query was selected and what it achieves.
- **Do not copy explanations unrelated to the question.** The explanation should match the exact code written.
- **Code + explanation is the safest format** when the question explicitly asks for explanation.